Kubernetes

Danijel Radaković

1 Uvod

Kubernetes

- Kubernetes (k8s) je platforma koja omogućava automatski deployment, skaliranje i upravljenje kontejnerima.
- Sve u kubernetesu je predstavljeno preko resursa koji imaju neku namenu.
  - Docker ima mali broj resursa: Service, Network, Volume, Secret, Config.
  - Kubernetes podržava mnogo više: Pod, Volume, Secret, ConfigMap, Service, Ingress, Job, CronJob, ReplicationSet, Deployment, StatefulSet itd.
  - Takođe podržava definisanje proizvoljnih resurasa (CRD).
    - Primer: AWS S3Bucket resurs kojim se konfigusire S3 Bucket. Kad se kreira resurs, njegova konfiguracija se prosleđuje kontroleru koji komunicira sa AWS API-om kako bi kreirao S3 Bucket u sladu sa konfiguracijom.
    - Kontoler i S3Bucket CRD je neophodno implementirati.

Alati i njihova konfiguracija

- Alat koji kreira Kubernetes klaster na lokalnoj mašini: k3d, Minikube, kind.
- kubectl - alat koji upravlja Kubernetes resursima.
- Sve slike koje se koriste treba da budu javne na nekom *container registry*-u (DockerHub) ili da se ubace u lokalni cluster:
  - k3d: `k3d image import`,
  - Minikube: `minikube image load`.

2 Kubernetes resursi

Resursi i objekti

- Resurs je manjen da reši neki problem koji se odnosi na hosting i održavanje sistema.
- Primeri:
  - Pod - pokreće kontejnerne.
  - Deployment - uvećava ili smanjuje replike.
  - Secret - čuva osetljive informacije.
- Objekti predtavljaju konkretne instance nekog resursa:
  - `pod/auth-server`
  - `pod/booking`,
  - `deployment/booking`.

Pod

- Pod je grupa od jednog ili više kontejnera koji dele storage (volumes) i mrežu. Sadrži specifikaciju kako pokrenuti kontejnere.
- Uglavnom Pod ima definisan samo jedan kontejner - samu aplikaciju.
  - Moguće je da Pod pored aplikacije ima definisane i pomoćne kontejnere: **sidecar** kontejneri (npr. log agent koji prikupja logove aplikacije i prosleđuje log agregatoru).
  - Moguće je da Pod pored aplikacija ima definisane i Init kontejnere (npr. kontejner koji će izvršiti migraciju baze, kontejner koji će kreairati Kafka topic-e, raditi provere pre pokretanja aplikacije).
  - Kontejneri unutar istog Pod-a mogu da komuniciraju preko *localhost*-a jer dele istu mrežu.

Pod


```yaml
# nginx.yml
apiVersion: v1
kind: Pod
metadata:
 name: nginx
spec:
 containers:
 - name: nginx
   image: nginx:1.14.2
   ports:
   - containerPort: 80
```

- Kreiranje: `kubectl apply -f nginx.yml`.
- Brisanje: `kubectl delete -f nginx.yml`.

Pod - CLI alternativa

- Pored `yaml` fajla može se direktno preko CLI-a kreirati Pod:

```
kubectl run --port 80 --image nginx:1.14.2 nginx
kubectl delete pod nginx
```

Healtcheck (liveness i readiness probes)


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: counter
spec:
containers:
  - name: counter
    image: danijelradakovic/counter
    ports:
      - containerPort: 8000
    livenessProbe:
      httpGet:
        port: 8000
        path: /probe/liveness # proverava da li je aplikacija živa na
```

Healtcheck (liveness i readiness probes) (ii)


```
slanjem GET zahteva na ovaj endpoint
      initialDelaySeconds: 3 # čekaj 3 sekunde pre slanja prvog zahteva
      periodSeconds: 3 # salje zahtev na svake 3 sekunde
    readinessProbe:
      httpGet:
        port: 8000
        path: /probe/readiness # proverava da li je aplikacija spremna da
prima saobracaj
      initialDelaySeconds: 3
      periodSeconds: 3
```

- Moramo da kreiramo namespace ukoliko ne postoji: `kubectl create namesapce demo`
- Kriranje: `kubectl -n demo apply -f pod.yaml`

Deployment

- Koristi se ukoliko je potrebno da postoje više replika istog Pod-a.
- Deployment vodi brigu da se broj željenih replika uvek bude ispunjen. Ukoliko neka replika Pod-a padne, Deployment će kreirati novu repliku unutar klastera.
- Broj replika je moguće povećavati ili smanjivati.

Deployment


```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: counter
 labels:
   app: counter
spec:
 replicas: 2 # broj replika
 selector:
   matchLabels:
     app: counter
 template:
   metadata:
```

Deployment (ii)


```yaml
    name: counter
    labels:
      app: counter
  spec:
    containers:
      - name: counter
        image: danijelradakovic/counter
        imagePullPolicy: IfNotPresent
        livenessProbe:
          httpGet:
            port: 8000
            path: /probe/liveness
          initialDelaySeconds: 3
          periodSeconds: 3
```

Deployment (iii)


```yaml
        readinessProbe:
          httpGet:
            port: 8000
            path: /probe/readiness
          initialDelaySeconds: 3
          periodSeconds: 3
      restartPolicy: Always
```

- `kubectl -n demo apply -f deployment.yaml`

Komunikacija između Pod-ova

- Kontejneri unutar istog Pod-a mogu komuniciranit preko `localhost:port`.
- Pod može da komunicira sa drugim Pod-om na osnovu IP adrese Pod-a.
- Adresa Pod-a nije statička, što znači da Pod može da dobije novu adresu kada se restartuje i prouzrokovati probleme ukoliko se komunikacija vrši preko IP adrese.
- Ukoliko postoji više replika istog Pod-a postavlja se pitanje kako će se vršiti balansiranje saobraćaja između replika?
- Rešenje za ove probleme nudi Service resurs.

Service

- Service *expose*-uje Pod ili Pod-ove tako da ostali Pod-ovi iz klustrera mogu da mu pristupe (neki servisi omogućavaju komunikaciju i van klustera).
- Service vrši:
  - Service discovery kojim omogućava da se Pod-ovima može pristupitu na osnovu naziva Service-a.
  - Load balancing nad jednim ili više Pod-ova ukoliko postoji više replika koji su deo jednog Deployment-a.

Service

- Komunikacija se uspostavlja:
  - Ukoliko su Pod-ovi iz istog namespace-a: `service_name:port`.
  - Ukoliko su Pod-ovi iz različitog namespace-a:
    - `service_name.namespace:port`,
    - `service_name.namespace.svc.cluster.local:port`.
- Koristi *liveness probe* kako bi registrovao da li je Pod živ.
- Koristi *readiness probe* kako bi odredio da li da pušta saobraćaj Pod-u.

Service


```yaml
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: counter
spec:
  selector:
    app: counter
  ports:
    - protocol: TCP
      port: 8000
      targetPort: 8000
```

- `kubectl -n demo apply -f service.yaml`

Service - CLI alternativa

- `kubectl -n demo expose deployment counter --port 8000`

Service - testiranje balansiranje sobraćaja


```
kubectl -n demo run -it --rm  --image curlimages/curl:8.00.1 curl -- sh

/ $ curl http://counter:8000
Counter:  10
/ $ curl http://counter:8000
Counter:  11
/ $ curl http://counter:8000
Counter:  8
/ $ curl http://counter:8000
Counter:  9
/ $ curl http://counter:8000
Counter:  12
/ $ curl http://counter:8000
Counter:  10
```

Service - testiranje balansiranje sobraćaja

- Moguće je koristiti IP adresu samog servisa:

```
kubectl -n counter describe service counter

Name:                     counter # counter se resolve-uje na: 10.99.55.145
Namespace:                counter
...
Selector:                 app=counter
Type:                     ClusterIP
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.99.55.145
IPs:                      10.99.55.145
Port:                     <unset>  8000/TCP
```

Service - testiranje balansiranje sobraćaja (ii)


```
TargetPort:        8000/TCP
Endpoints:         10.244.0.4:8000,10.244.0.5:8000 # IP adrese i portovi
pod-ova obuhvaćeni servisom
...

/$ curl http://10.99.55.145:8000
Counter:  11
```

Service - testiranje komunikacije iz drugog namespace-a


```
kubectl create namespace tmp
kubectl -n tmp run -it --rm  --image curlimages/curl:8.00.1 curl -- sh

/ $ curl http://counter.demo:8000
Counter:  16
/ $ curl http://10.99.55.145:8000
Counter:  17
```

Ingress

- Service nam omogućava da radimo komunikaciju unutar klastera. Kako omogućiti da neko van klustera može da pristupi našoj aplikaciji?
- Za to se koristi Ingress resurs.
- **Ingress *expose*-uje samo HTTP (80) i HTTPS (443) portove!**
- Ukoliko imate potrebu da otvorite neki drugi port na klasteru koristite Service tipa NodePort ili LoadBalancer.

Ingress

- Ingress resurs zavisi od samog Kubernetes klastera gde se hostuje. Drugačije se ponaša na GKE, EKS, k3d, Minikube itd.
- U suštini Ingress je nadležan da kreira LoadBalancer koji će preusmeravati dolazni saobraćaj na port 80 ili 443. Dalje se saobraćaj prosleđuje na odgovarajući Service koji dalje rutira saobraćaj do odgovarajućeg Pod-a.
- Postoji Ingress operator koji radi kreiranje i konfiguraciju za vas.

Ingress - Echo aplikacija


```yaml
# echo-server.yaml
kind: Pod
apiVersion: v1
metadata:
  name: echo
labels:
  app: echo
spec:
  containers:
    - name: echo
      image: 'kicbase/echo-server:1.0'
---
kind: Service
apiVersion: v1
```

Ingress - Echo aplikacija (ii)


```yaml
metadata:
  name: echo
spec:
  selector:
    app: echo
  ports:
    - port: 8080
```

- `kubectl -n demo apply -f echo-server.yaml`

Ingress


```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
 name: demo
spec:
 rules:
   - http:
       paths:
         - pathType: Prefix
           path: /counter
           backend:
             service:
               name: counter
               port:
```

Ingress (ii)


```yaml
                number: 8000
        - pathType: Prefix
          path: /echo
          backend:
            service:
              name: echo
              port:
                number: 8080
```

- `kubectl -n demo apply -f ingress.yaml`

Ingress


```
kubectl -n demo describe ingress demo

Name:             demo
Labels:           <none>
Namespace:        demo
Address:          192.168.49.2
Ingress Class:    nginx
Default backend:  <default>
Rules:
 Host        Path  Backends
 ----        ----  --------
 *
             /counter   counter:8000 (10.244.0.83:8000,10.244.0.85:8000)
             /echo      echo:8080 (10.244.0.91:8080)
```

Ingress


```
curl http://192.168.49.2/echo

Request served by echo
HTTP/1.1 GET /echo
Host: 192.168.49.2
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/
avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding: gzip, deflate
```

```
curl http://192.168.49.2/counter

Counter: 17
```

Ingress


```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: counter
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - pathType: Prefix
            path: /counter
            backend:
              service:
                name: counter
                port:
                  number: 8000
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: echo
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - pathType: Prefix
            path: /echo
            backend:
              service:
                name: echo
                port:
                  number: 8080
```

Slika 1: Razdvajanje Ingress resursa

Scope resurasa

- Resursi mogu biti **namespace-scoped** ili **cluster-wide**.
- Namespace-scoped resursi moraju da budu deo nekog namespace-a. Primeri: Pod, Deployment, ConfigMap, Secret, Ingress itd.
- Nazivi objekata nekog resursa unutar namespace-a moraju da budu jedinstveni, ali ne moraju u drugim namespace-ovima.
- Cluster-wide resursi ne pripadaju nijednom namespace-u i nazivi objekata nekog resursa moraju da budu jedinstveni na nivou čitavog klastera.

ConfigMap

- Koristi se za kreiranje konfiguracija za aplikacije koje se pokreću preko Pod-ova.
- Pod-ovi mogu da koriste ConfigMap-u kao enviroment varijable, argumente komandne linije ili kao konfiguracione fajlove.
- ConfigMap-a ima `data` sekciju koja se sastoji od kolekcije kljuc-vrednost parova. Šta predstavlja ključ a šta vrednost zavisi kako se ConfigMap-a koristi.
- Kako automatski uraditi restart poda kase se izmeni ConfigmMap-a

ConfigMap - environment varijable


```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: booking
data:
  LOG_LEVEL: "debug"
  FETCH_TIMEOUT: "1m"
```

ConfigMap - environment varijable


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: booking
spec:
  containers:
    - name: app
      image: alpine
      envFrom:
        - configMapRef:
            name: booking
```

ConfigMap - environment varijable

- Ukoliko ne želimo sve key-value parove da koristimo, možemo selektujemo samo one parove koju su potrebni.
- Takođe imamo opciju da promenimo naziv env varijable.

ConfigMap - environment varijable


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: booking
spec:
  containers:
    - name: app
      image: alpine
      env:
        - name: HTTP_TIMEOUT
          valueFrom:
            configMapKeyRef:
              name: booking
              key: FETCH_TIMEOUT
```

ConfigMap - argumenti komande linije


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: booking
spec:
  containers:
    - name: app
      image: alpine
      envFrom:
        - configMapRef:
            name: booking
      command: ["/bin/sh", "-c"]
      args: ["echo $FETCH_TIMEOUT"]
```

ConfigMap - konfiguracioni fajlovi


```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: booking
data:
  dev.toml: |
    mode=dev
    timeout=1m
```

ConfigMap - konfiguracioni fajlovi


```yaml
...
spec:
  containers:
    - name: app
      image: alpine
      command: ["/bin/app"]
      args: ["--config /etc/booking/dev.toml"]
      volumeMounts:
      - name: config
        mountPath: /etc/booking
  volumes:
    - name: config
      configMap:
        name: booking
```

ConfigMap - konfiguracioni fajlovi

- Imamo dva konfiguraciona fajla u ConfigMap-i, želimo da koristimo samo jedan.

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: booking
data:
  dev.toml: |
    mode=dev
    timeout=1m
  prod.toml: |
    mode=prod
    timeout=90s
```

ConfigMap - konfiguracioni fajlovi, `subPath` koncept


```yaml
spec:
  containers:
    - name: app
      image: alpine
      command: ["/bin/app"]
      args: ["--config /etc/booking/config.toml"]
      volumeMounts:
      - name: config
        mountPath: /etc/booking/config.toml # Putanja unutar kontejnera
        subPath: dev.toml  # Ključ (fajl) iz ConfigMap-e
  volumes:
    - name: config
      configMap:
        name: booking
```

ConfigMap - konfiguracioni fajlovi, `items` koncept


```yaml
spec:
  containers:
    - name: app
      ...
      volumeMounts:
      - name: config
        mountPath: /etc/booking # --> /etc/booking/config.toml
  volumes:
    - name: config
      configMap:
        name: booking
        items:
          - key: "dev.toml" # Ključ (fajl) iz ConfigMap-e
            path: "config.toml" # Naziv fajla u kontejneru
```

ConfigMap - automatsko ažuriranje

- Mounted CoinfigMap-e se automatski ažuriraju kada nastanu izmene. Samim tim nema potrebe da se restartuje Pod.
- Ukoliko se koristi `subPath`, **gubi se** mehanizam automatskog ažuriranja, dok korišćenje `items` omogućava automatsko ažuriranje.
- Aplikacije koje hoće da koriste ovaj mehanizam moraju da implementiraju u pozadini tred koji će gledati promene fajla.
- **Napomena**: ukoliko se podesi `mountPoint` bez `subPath`-a, sav sadržaj folder unutar kontejnera će biti pregažen i unutar tog folder će se nalaziti fajlovi definisani u ConfigMap-i.

ConfigMap - primer automatskog ažuriranja


```yaml
# config.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: booking
data:
  dev.toml: |
    mode=dev
    timeout=1m
```

ConfigMap - primer automatskog ažuriranja


```yaml
# app.yaml
apiVersion: v1
kind: Pod
metadata:
  name: demo
spec:
  containers:
    - name: alpine
      image: alpine
      command: ["sleep", "infinity"]
      volumeMounts:
        - name: config
          mountPath: /etc/booking
  volumes: [{"name": "config", "configMap": {"name": "booking"}}]
```

ConfigMap - primer automatskog ažuriranja


```
kubectl apply -f config.yaml
kubectl apply -f app.yaml
kubect  exec -it demo -- ash
watch -n 1 "ls -l /etc/booking && echo --- && cat /etc/booking/dev.toml"
```

ConfigMap - primer automatskog ažuriranja


```
Every 1.0s: ls -l /etc/booking && echo --- && cat /etc/booking/dev.toml
2026-04-19 17:32:06

total 0
lrwxrwxrwx    1 root     root            15 Apr 19 17:27 dev.toml -
> ..data/dev.toml
---
mode=dev
timeout=1m
```

```
kubect edit configmap booking
```

ConfigMap - primer automatskog ažuriranja


```
Every 1.0s: ls -l /etc/booking && echo --- && cat /etc/booking/dev.toml
2026-04-19 17:37:44

total 0
lrwxrwxrwx    1 root     root            15 Apr 19 17:27 dev.toml -
> ..data/dev.toml
---
mode=prod  <---- changed this line
timeout=1m
```

ConfigMap - automatsko ažuriranje

- Postoje latency lag prilikom ažuriranja na koji treba obratiti pažnju.
- Na letency utiču 3 stvare:
  - Propagation Delay: Vreme koje je potrebno da Kubernetesov API server notifikuje ostale komponente klastera da se izmena ConfigMap-e desila.
  - Kubelet Sync Period: Ovo je najveći lag. Kubelet na svakog nodu izvršava sync loop (podrazuvemano 1 minut) kako bi proverio da li se trenutni volume podudara sa željenim stanjem.
  - Cache TTL: Kubelet ponekad ne komunicira direktno sa API serverom, jer koristi lokalni keš i može se konfigurisati pomoću `configMapAndSecretChangeDetectionStrategy`.
- Može se koristiti u nekritičnim komponentama sistema. Za kritične komponente, koristiti klasičan način ažuriranja.

ConfigMap

- ConfigMap-a i Pod moraju da budu u istom namepsace-u. Pod ne može koristiti ConfigMap-e iz drugog namespace-a.
- Ukoliko Pod treba da koristi ConfigMap-u iz drugog namespace-a, ConfigMap-a se mora kopirati u namespace Pod-a.
- Postoje i Immutable ConfigMap koja se ne može menjati, nego samo obrisati.

3 Primeri

Aplikacije

- Pogledati primer Dojo aplikacije koja koristi PostgreSQL, Ingress, Helm i podešen monitoring.
- Za vizualizaciju objekata unutar klastera može se koristiti: Headlamp